ARTIFICIAL INTELLIGENCE & MACHINE LEARNING LAB 2

Final Practical Exam Preparation Document

Simple theory • University-aligned concepts • Algorithms • Complete Python code • Viva questions

Course: PCC-305-ITT | **Class:** TE-A | **Academic Year:** 2026–27

Prepared according to the university/college List of Experiments supplied (3 pages).

How to use this guide

For the written practical exam, learn the Aim, Theory, Key Concepts and Algorithm. Then practise writing the code and understand what each major block does. The viva questions at the end of each practical are based on the experiment topic.

Important alignment note

The supplied university list contains 10 experiments. Your ZIP archive contains code for Experiments 1–9 and the three supplied datasets. Experiment 10 is a project/application experiment; this guide includes a complete Customer Segmentation implementation using the supplied Mall_Customers.csv dataset as one of the application choices listed by the university.

Install required libraries

```
pip install numpy pandas matplotlib seaborn scikit-learn scipy
```

Keep the CSV files in the same folder as the Python script: MarvellousTitanicDataset.csv, california_housing.csv and Mall_Customers.csv. For Google Colab, upload the required CSV file or mount Drive and update the path.

Experiment 1: Intelligent Agents and PEAS Representation

Aim

To design and implement a simple intelligent agent for a smart-home application and represent its task using PEAS.

Theory in simple words

Extra concepts / quick revision

An intelligent agent is a system that observes its surroundings and takes action to achieve a goal. It receives information through sensors and performs actions through actuators. Example: a smart-home controller reads temperature and motion, then switches a fan, light or alarm. A simple reflex agent chooses an action using only the current input. It follows condition–action rules such as: IF temperature is above 30°C, THEN turn the fan ON. It does not remember earlier inputs. PEAS describes an agent's task: • P – Performance Measure: how success is judged (comfort, safety, low energy use). • E – Environment: where the agent works (rooms, people, appliances). • A – Actuators: how it acts (fan switch, light switch, alarm). • S – Sensors: how it observes (temperature, motion and light sensors). PEAS for this smart home: Performance: maintain comfort and safety while saving energy. Environment: house rooms, occupants and appliances. Actuators: fan, light and alarm. Sensors: temperature sensor, motion sensor, light sensor and owner-presence input. Agent types: simple reflex (current percept), model-based (uses internal state), goal-based (chooses actions to reach a goal), utility-based (chooses the most useful action), and learning agent (improves from experience). In this program, the function smart_home_agent() applies three rules and returns the status of the fan, light and alarm.

Algorithm / Procedure

1. Identify the task using PEAS. 2. Read sensor values. 3. Apply IF–THEN rules. 4. Send actions to actuators. 5. Display device states.

Complete Python Code

```
# ============================================================
# PRACTICAL 1 - SMART HOME AGENT
# Simple Reflex Agent
# ============================================================

# ---------- AGENT FUNCTION ----------

def smart_home_agent(temperature, motion, light, owner_home):

    actions = {}

    # Rule 1: Cooling
    if temperature > 30:
        actions["Fan"] = "ON"
    else:
        actions["Fan"] = "OFF"

    # Rule 2: Lighting
    if motion == "yes" and light == "dark":
        actions["Light"] = "ON"
    else:
        actions["Light"] = "OFF"

    # Rule 3: Security
    if motion == "yes" and owner_home == "no":
        actions["Alarm"] = "ON"
    else:
        actions["Alarm"] = "OFF"

    return actions


# ---------- SENSORS ----------

print("--- SENSOR READINGS ---")

temperature = int(input("Temperature sensor (Celsius): "))
motion = input("Motion sensor (Yes/No): ").lower()
light = input("Light sensor (Light/Dark): ").lower()
owner_home = input("Owner home (Yes/No): ").lower()


# ---------- AGENT ----------

decisions = smart_home_agent(
    temperature,
    motion,
    light,
    owner_home
)


# ---------- ACTUATORS ----------

print("\n---- ACTUATOR STATUS ----")

for device in decisions:
    print(device, "=", decisions[device])
```

Viva questions to practise

What is an agent? What are sensors and actuators? Expand PEAS. What is a simple reflex agent? Give one IF–THEN rule.

Experiment 2: Uninformed Search Algorithms – BFS and DFS

Aim

To implement BFS and DFS for route finding and compare path length, explored nodes and execution time.

Theory in simple words

Extra concepts / quick revision

A search problem contains an initial state, a goal state, possible actions and a path from one state to another. In route finding, locations are nodes and roads are edges. Breadth First Search (BFS) explores the graph level by level. It uses a FIFO queue: First In, First Out. It visits all nearby nodes before going farther. In an unweighted graph, BFS finds a path with the minimum number of edges. Depth First Search (DFS) explores one branch as far as possible before backtracking. It uses a LIFO stack: Last In, First Out. DFS may find a path quickly, but it does not guarantee the shortest path. Comparison: BFS – queue; level-wise; shortest path in an unweighted graph; usually needs more memory. DFS – stack; depth-wise; does not guarantee shortest path; often uses less memory. For a graph with V vertices and E edges, both implementations take O(V + E) time with an adjacency list. The program compares path length, number of explored nodes and execution time. Execution time can vary between runs and depends on the machine.

Algorithm / Procedure

1. Represent locations as graph nodes and connections as edges. 2. Run BFS with a queue. 3. Run DFS with a stack. 4. Reconstruct each route. 5. Compare path length, explored nodes and execution time.

Complete Python Code

```
from collections import deque
import time

graph = {
'A': ['B', 'C'],
'B': ['A', 'D', 'E'],
'C': ['A', 'F'],
'D': ['B'],
'E': ['B', 'F'],
'F': ['C', 'E', 'G'],
'G': ['F']
}

start = 'A'

goal = 'G'

def reconstruct_path(parent, goal):
    if goal not in parent:
        return []

    path = []
    current = goal

    while current is not None:
        path.append(current)
        current = parent[current]

    path.reverse()
    return path

def bfs(graph, start, goal):
    queue = deque([start])
    visited = {start}
    parent = {start: None}
    nodes_explored = 0

    while queue:
        current = queue.popleft()
        nodes_explored += 1

        if current == goal:
            path = reconstruct_path(parent, goal)
            return path, nodes_explored

        for neighbor in graph[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = current
                queue.append(neighbor)

    return [], nodes_explored

bfs_path, bfs_nodes = bfs(graph, start, goal)
print("BFS Path:", bfs_path)
print("BFS Path Length:", len(bfs_path) - 1)
print("BFS Nodes Explored:", bfs_nodes)

def dfs(graph, start, goal):
    stack = [start]
    visited = {start}
    parent = {start: None}
    nodes_explored = 0
```

```
    while stack:
        current = stack.pop()
        nodes_explored += 1

        if current == goal:
            path = reconstruct_path(parent, goal)
            return path, nodes_explored

        for neighbor in reversed(graph[current]):
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = current
                stack.append(neighbor)

    return [], nodes_explored

dfs_path, dfs_nodes = dfs(graph, start, goal)
print("DFS Path:", dfs_path)
print("DFS Path Length:", len(dfs_path) - 1)
print("DFS Nodes Explored:", dfs_nodes)

# --------- BFS Timing -----------

start_time = time.perf_counter()
bfs_path, bfs_nodes = bfs(graph, start, goal)
bfs_time = time.perf_counter() - start_time
bfs_length = len(bfs_path) - 1
print("BFS Path:", bfs_path)
print("BFS Path Length:", bfs_length)
print("BFS Nodes Explored:", bfs_nodes)
print("BFS Execution Time:", bfs_time, "seconds")

# ---------- DFS Timing ------------

start_time = time.perf_counter()
dfs_path, dfs_nodes = dfs(graph, start, goal)
dfs_time = time.perf_counter() - start_time
dfs_length = len(dfs_path) - 1
print("DFS Path:", dfs_path)
print("DFS Path Length:", dfs_length)
print("DFS Nodes Explored:", dfs_nodes)
print("DFS Execution Time:", dfs_time, "seconds")
```

Viva questions to practise

What is a state space? Which data structure does BFS use? Which does DFS use? Which finds the shortest unweighted path? What do nodes explored and path length mean?

Experiment 3: Knowledge Representation and Expert System – Forward and Backward Chaining

Aim

To develop a rule-based expert system using propositional logic with Forward Chaining and Backward Chaining.

Theory in simple words

Extra concepts / quick revision

Knowledge representation stores facts and rules so a computer can reason. A rule-based expert system imitates a human expert in a limited subject area. It contains a knowledge base (facts and IF–THEN rules) and an inference engine (the part that applies rules). Propositional logic represents statements that are either TRUE or FALSE. Example: Python is known, Mathematics is known. A rule can be written as: IF Python AND Mathematics, THEN Software Engineering. Forward Chaining is data-driven. It starts with known facts, checks which rules match, adds new conclusions and repeats until no more facts can be derived or the required conclusion is found. Backward Chaining is goal-driven. It starts with a question (goal), searches for a rule that can produce that goal, and then tries to prove the rule's conditions. It succeeds if all required conditions are known or can themselves be proved. Example knowledge rule: IF mathematics AND problem_solving, THEN data_science. If both starting facts are available, forward chaining derives data_science. Backward chaining can start with the goal data_science and ask whether both conditions are true. This program uses skills as facts and career-related outcomes as conclusions. It prints the rules applied and whether the selected goal can be proved.

Algorithm / Procedure

1. Store facts and IF–THEN rules. 2. Read user facts and goal. 3. Forward chain: repeatedly fire rules whose conditions are known. 4. Backward chain: start from goal and prove its conditions. 5. Display conclusions.

Complete Python Code

```
# KNOWLEDGE BASE: IF conditions are true, THEN conclusion is true.
rules = [
    (["python", "mathematics"], "software_engineering"),
    (["mathematics", "problem_solving"], "data_science"),
    (["data_science", "communication"], "data_analyst"),
    (["python", "hardware_interest"], "embedded_systems"),
    (["software_engineering", "communication"], "technical_lead")
]

def forward_chaining(facts, rules):
    """Keep applying rules until no new fact can be added."""
    known = set(facts)
    changed = True
    while changed:
        changed = False
        for conditions, conclusion in rules:
            if all(condition in known for condition in conditions):
                if conclusion not in known:
                    known.add(conclusion)
                    print("Rule fired:", conditions, "->", conclusion)
                    changed = True
    return known

def backward_chaining(goal, facts, rules, depth=0):
    """Start from a goal and recursively try to prove its conditions."""
    indent = " " * depth
    print(indent + "Goal:", goal)
    if goal in facts:
        print(indent + "Known fact: TRUE")
        return True
    for conditions, conclusion in rules:
        if conclusion == goal:
            print(indent + "Trying rule:", conditions, "->", conclusion)
            if all(backward_chaining(c, facts, rules, depth + 2)
                   for c in conditions):
                print(indent + "Proved:", goal)
                return True
    print(indent + "Cannot prove:", goal)
    return False

user_input = input("Enter your skills (comma separated): ")
facts = {skill.strip().lower() for skill in user_input.split(",") if skill.strip()}
goal = input("Enter the goal to check (e.g. data_science): ").strip().lower()

print("\n--- FORWARD CHAINING ---")
all_facts = forward_chaining(facts, rules)
print("Facts derived:", sorted(all_facts))
print("\n--- BACKWARD CHAINING ---")
result = backward_chaining(goal, facts, rules)
print("\nResult:", goal, "is", "PROVED" if result else "NOT PROVED")
```

Viva questions to practise

What is a knowledge base? What is an inference engine? What is a propositional statement? Differentiate forward and backward chaining.

Experiment 4: Data Preprocessing and Exploratory Data Analysis

Aim

To preprocess a standard dataset using cleaning, missing-value handling, scaling, encoding, statistical analysis, visualization and train-test splitting.

Theory in simple words

Extra concepts / quick revision

A dataset is a collection of records (rows) and attributes (columns). Real-world data may contain missing values, irrelevant columns, inconsistent formats and values on very different scales. Preprocessing prepares data before machine learning. Data cleaning removes or corrects unwanted data. Missing-value handling fills empty numeric values with a mean or median and categorical values with the mode. Median is often safer when numeric data contains extreme values. Feature scaling brings numeric columns to a similar scale. StandardScaler converts a value using z = (x – mean) / standard deviation. Encoding converts text categories into numbers. Label Encoding assigns an integer to each category. EDA means Exploratory Data Analysis. It uses summaries and graphs to understand the dataset. describe() gives count, mean, standard deviation, minimum, quartiles and maximum. A histogram shows the distribution of a numeric variable. A boxplot shows median, quartiles, spread and possible outliers. A train-test split separates data for model building and evaluation. The training set is used to learn; the testing set is kept aside to check performance on unseen records. This practical uses the supplied Titanic CSV: removes Cabin, fills missing Age and Embarked values, scales Fare, encodes Sex and passenger class, displays statistics and graphs, then splits the data.

Algorithm / Procedure

1. Load Titanic CSV. 2. Inspect and clean columns. 3. Fill missing numeric/categorical values. 4. Scale Fare and encode categories. 5. Display statistics, histogram and boxplot. 6. Split into train and test sets.

Complete Python Code

```
# TITANIC DATASET - DATA PREPROCESSING

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split


# 1. Load Dataset from CSV
titanic = pd.read_csv("MarvellousTitanicDataset.csv")

print("First 5 rows of dataset:")
print(titanic.head())


# 2. Data Cleaning
# Drop irrelevant columns
titanic = titanic.drop(['Cabin'], axis=1)


# 3. Missing Value Handling
# Fill numeric missing values with median
titanic['Age'] = titanic['Age'].fillna(titanic['Age'].median())

# Fill categorical missing values with mode
titanic['Embarked'] = titanic['Embarked'].fillna(
    titanic['Embarked'].mode()[0]
)


# 4. Feature Scaling
scaler = StandardScaler()

titanic['Fare_scaled'] = scaler.fit_transform(
    titanic[['Fare']]
)


# 5. Encoding Techniques
encoder = LabelEncoder()

titanic['Sex_encoded'] = encoder.fit_transform(titanic['Sex'])

titanic['Class_encoded'] = encoder.fit_transform(titanic['Pclass'])


# 6. Statistical Analysis
print("\nStatistical Analysis:")
print(titanic.describe())


# 7. Data Visualization

# Histogram of Age
sns.histplot(titanic["Age"])
plt.title("Age Distribution")
```

```
plt.show()

# Boxplot of Fare
sns.boxplot(x=titanic["Fare"])
plt.title("Fare Distribution")
plt.show()


# 8. Train-Test Split

X = titanic[['Age', 'Fare_scaled', 'Sex_encoded', 'Class_encoded']]

y = titanic['Survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


# 9. Display Train-Test Shapes
print("\nTraining Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)

print("\nData Preprocessing Completed Successfully!")
```

Viva questions to practise

What is preprocessing? Why handle missing values? Mean vs median vs mode? What is StandardScaler? What do histogram and boxplot show? Why split train and test?

Experiment 5: Regression Techniques – Linear and Polynomial Regression

Aim

To implement Linear and Polynomial Regression and compare their performance using MSE, MAE and R^2.

Theory in simple words

Extra concepts / quick revision

Regression is a supervised machine-learning technique used to predict a continuous number, such as a house price. Linear Regression fits a straight-line relationship between input features and the target. For one input, the equation is y = mx + c, where m is the slope and c is the intercept. For multiple inputs, the model combines several feature values. Polynomial Regression adds powers of the input features (such as x^2) so the model can represent a curved relationship. PolynomialFeatures creates these extra columns; LinearRegression learns their coefficients. The dataset is divided into training and testing parts. The model learns from training data and predicts values for the test data. Evaluation metrics: • MSE = average of squared errors. Lower is better; large errors are penalized strongly. • MAE = average absolute error. It is easy to understand in the target's units. • R^2 = proportion of target variation explained by the model. A value closer to 1 is generally better; it can be negative when the model performs poorly. The practical uses California Housing data and compares Linear Regression with degree-2 Polynomial Regression using MSE, MAE and R^2. Polynomial models are not always better; they may overfit.

Algorithm / Procedure

1. Load California Housing CSV. 2. Inspect and handle missing values. 3. Encode ocean_proximity. 4. Separate features and target. 5. Split data. 6. Train linear and polynomial models. 7. Compare MSE, MAE and R^2.

Complete Python Code

```
# ============================================================
# AIML PRACTICAL 2 - REGRESSION TECHNIQUES
# Linear Regression and Polynomial Regression
# Dataset: California Housing Dataset (housing.csv)
# ============================================================

# 1. IMPORTING REQUIRED LIBRARIES

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score


# 2. LOADING THE DATASET

df = pd.read_csv("california_housing.csv")

print("First 5 rows:")
print(df.head())

print("\nLast 5 rows:")
print(df.tail())


# 3. DATA PREPROCESSING

# Checking dimensions
print("\nDataset Shape:")
print(df.shape)

# Printing information
print("\nDataset Information:")
df.info()

# Descriptive statistics
print("\nStatistical Description:")
print(df.describe())

# Checking missing values
print("\nMissing Values:")
print(df.isnull().sum())


# 4. HANDLING MISSING VALUES

df['total_bedrooms'] = df['total_bedrooms'].fillna(
    df['total_bedrooms'].median()
)


# 5. DETERMINING CATEGORICAL FEATURES

print("\nOcean Proximity:")
print(df['ocean_proximity'].value_counts())
```

```
# 6. ENCODING CATEGORICAL FEATURES

ohe = OneHotEncoder(
    sparse_output=False,
    drop='first'
)

encoded = ohe.fit_transform(df[['ocean_proximity']])

encoded_df = pd.DataFrame(
    encoded,
    columns=ohe.get_feature_names_out(['ocean_proximity']),
    index=df.index
)

df = pd.concat(
    [df.drop('ocean_proximity', axis=1), encoded_df],
    axis=1
)

print("\nDataset After Encoding:")
print(df.head())


# 7. EXPLORATORY DATA ANALYSIS

# Finding pairwise correlation
print("\nCorrelation Matrix:")
print(df.corr(numeric_only=True))

# Generating correlation heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()


# 8. SCATTER PLOT

sns.scatterplot(
    x='median_income',
    y='median_house_value',
    data=df
)

plt.title("Median Income vs Median House Value")
plt.show()


# 9. BUILDING LINEAR REGRESSION MODEL

# Extracting independent and dependent variables

X = df.drop('median_house_value', axis=1)
y = df['median_house_value']


# Splitting dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)


# Creating Linear Regression model

lin_reg = LinearRegression()


# Fitting Linear Regression model

lin_reg.fit(X_train, y_train)


# Predicting test set results

y_pred = lin_reg.predict(X_test)


# 10. EVALUATING LINEAR REGRESSION

print("\n========== LINEAR REGRESSION ==========")

print("MSE:", mean_squared_error(y_test, y_pred))

print("MAE:", mean_absolute_error(y_test, y_pred))

print("R2 Score:", r2_score(y_test, y_pred))


# 11. VISUALIZATION OF LINEAR REGRESSION

plt.figure(figsize=(8, 5))

sns.scatterplot(
    x=X_test['median_income'],
    y=y_test,
    alpha=0.3,
    label="Test Data"
)

sns.lineplot(
    x=X_test['median_income'],
    y=y_pred,
```

```
    color='red',
    label="Predicted Values"
)

plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Linear Regression Visualization")
plt.show()


# 12. POLYNOMIAL REGRESSION

# Creating Polynomial Features

poly = PolynomialFeatures(degree=2)

X_train_poly = poly.fit_transform(X_train)

X_test_poly = poly.transform(X_test)


# Creating Polynomial Regression model

poly_reg = LinearRegression()


# Fitting Polynomial Regression model

poly_reg.fit(X_train_poly, y_train)


# Predicting test set results

y_pred_poly = poly_reg.predict(X_test_poly)


# 13. EVALUATING POLYNOMIAL REGRESSION

print("\n========== POLYNOMIAL REGRESSION ==========")

print("MSE:", mean_squared_error(y_test, y_pred_poly))

print("MAE:", mean_absolute_error(y_test, y_pred_poly))

print("R2 Score:", r2_score(y_test, y_pred_poly))


# 14. COMPARISON OF BOTH MODELS

print("\n========== MODEL COMPARISON ==========")

print("\nLinear Regression:")
print("MSE:", mean_squared_error(y_test, y_pred))
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

print("\nPolynomial Regression:")
print("MSE:", mean_squared_error(y_test, y_pred_poly))
print("MAE:", mean_absolute_error(y_test, y_pred_poly))
print("R2 Score:", r2_score(y_test, y_pred_poly))


# 15. CONCLUSION

print("\nConclusion:")
print("Thus, we have studied Regression Techniques and")
print("implemented Linear Regression and Polynomial Regression.")
```

Viva questions to practise

What is regression? What is the difference between linear and polynomial regression? Define MSE, MAE and R^2. What is overfitting?

Experiment 6: Classification using Decision Tree and Naïve Bayes

Aim

To implement Decision Tree and Naïve Bayes classifiers and compare their performance using evaluation metrics.

Theory in simple words

Extra concepts / quick revision

Classification is supervised learning in which the model predicts a class/category. In this practical, the Titanic target is Survived: 0 means did not survive and 1 means survived. A Decision Tree makes predictions by asking a sequence of feature-based questions. Each internal node is a test, each branch is an answer, and each leaf is the final class. It is easy to understand and can model nonlinear patterns. Entropy and Gini are common measures used to choose splits. Naïve Bayes uses Bayes' theorem to calculate the probability of a class given the input features. It is called naïve because it assumes features are conditionally independent when the class is known. Gaussian Naïve Bayes is commonly used for numeric features that are treated as normally distributed. The program cleans the Titanic data, converts categories into numeric values, splits the data, trains both classifiers and compares their results. Evaluation: • Confusion matrix: counts correct and incorrect predictions. • Accuracy: correct predictions / all predictions. • Precision: of predicted positives, how many are truly positive. • Recall: of actual positives, how many were found. • F1-score: harmonic mean of precision and recall. A suitable model should be selected using the metric that matters for the problem, not accuracy alone.

Algorithm / Procedure

1. Load Titanic CSV. 2. Clean missing values and remove irrelevant columns. 3. Encode categories. 4. Split into train and test. 5. Train Decision Tree and Gaussian Naïve Bayes. 6. Compare evaluation metrics.

Complete Python Code

```
# ============================================================
# AIML LAB - ASSIGNMENT 3
# CLASSIFICATION USING DECISION TREE AND NAIVE BAYES
# Dataset: Titanic-Dataset.csv
# ============================================================

# STEP 1: IMPORTING REQUIRED LIBRARIES

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    precision_score,
    recall_score,
    fbeta_score
)

# ============================================================
# STEP 2: LOADING THE DATASET
# ============================================================

titanic = pd.read_csv("MarvellousTitanicDataset.csv")

print("First 5 rows:")
print(titanic.head())

# ============================================================
# STEP 3: DATA PREPARATION
# ============================================================

print("\nFirst 5 rows:")
print(titanic.head())

print("\nLast 5 rows:")
print(titanic.tail())

print("\nDimensions of dataset:")
print(titanic.shape)

print("\nDataset Information:")
titanic.info()

print("\nDescriptive Statistics:")
print(titanic.describe())

print("\nMissing Values:")
print(titanic.isnull().sum())

print("\nDuplicate Records:")
print(titanic.duplicated().sum())
```

```
# ============================================================
# STEP 4: DATA PREPROCESSING
# ============================================================

# Handling missing values

titanic['Age'] = titanic['Age'].fillna(
    titanic['Age'].mean()
)

titanic['Embarked'] = titanic['Embarked'].fillna(
    titanic['Embarked'].mode()[0]
)

titanic['Fare'] = titanic['Fare'].fillna(
    titanic['Fare'].mean()
)

# Checking unique values

print("\nUnique values in each column:")
print(titanic.nunique())

# Dropping unnecessary columns

titanic.drop(
    columns=['PassengerId', 'Name', 'Ticket', 'Cabin'],
    inplace=True
)

# Converting categorical variables into numerical values

le = LabelEncoder()

titanic['Sex'] = le.fit_transform(titanic['Sex'])

titanic['Embarked'] = le.fit_transform(
    titanic['Embarked']
)

print("\nProcessed Dataset:")
print(titanic.head())

# ============================================================
# STEP 5: EXPLORATORY DATA ANALYSIS (EDA)
# ============================================================

# Check how many passengers survived

sns.countplot(x=titanic['Survived'])
plt.title("Number of Passengers Survived")
plt.show()

# Compare survival by gender

sns.countplot(
    x=titanic['Survived'],
    hue=titanic['Sex']
)
plt.title("Survival by Gender")
plt.show()

# Compare survival by passenger class

sns.countplot(
    x=titanic['Pclass'],
    hue=titanic['Survived']
)
plt.title("Survival by Passenger Class")
plt.show()

# Analyze relationship between age and survival

sns.barplot(
    x=titanic['Survived'],
    y=titanic['Age']
)
plt.title("Age and Survival")
plt.show()

# Analyze fare and survival

sns.barplot(
    x=titanic['Survived'],
    y=titanic['Fare']
)
plt.title("Fare and Survival")
plt.show()

# Survival by gender and passenger class

sns.barplot(
    x=titanic['Pclass'],
    y=titanic['Survived'],
    hue=titanic['Sex']
)
plt.title("Survival by Gender and Passenger Class")
plt.show()

sns.catplot(
    x="Pclass",
    hue="Survived",
    col="Sex",
    data=titanic,
```

```
    kind="count"
)
plt.show()

# Survival by embarkation port

sns.countplot(
    x=titanic['Survived'],
    hue=titanic['Embarked']
)
plt.title("Survival by Embarkation Port")
plt.show()

# ============================================================
# STEP 6: EXTRACTING FEATURES AND TARGET
# ============================================================

X = titanic.drop("Survived", axis=1)

y = titanic['Survived']

print("\nFeatures:")
print(X.head())

print("\nTarget:")
print(y.head())

# ============================================================
# STEP 7: TRAIN-TEST SPLIT
# ============================================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# ============================================================
# STEP 8: FEATURE SCALING
# ============================================================

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

# ============================================================
# STEP 9: DECISION TREE CLASSIFIER
# ============================================================

# Build the classification model

dt = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=4,
    random_state=42
)

dt.fit(X_train, y_train)

# Make predictions

y_pred_dt = dt.predict(X_test)

# Evaluate the model

print("\n========== DECISION TREE RESULTS ==========")

print(
    "Decision Tree Accuracy:",
    accuracy_score(y_test, y_pred_dt)
)

print("\nClassification Report:")
print(classification_report(y_test, y_pred_dt))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_dt))

# Plot the decision tree

plt.figure(figsize=(16, 10))

plot_tree(
    dt,
    feature_names=X.columns,
    class_names=['Not Survived', 'Survived'],
    filled=True
)

plt.title("Decision Tree Classifier")
plt.show()

# ============================================================
# STEP 10: NAIVE BAYES CLASSIFIER
# ============================================================

# Build the classification model

nb = GaussianNB()

nb.fit(X_train, y_train)
```

```
# Make predictions

y_pred_nb = nb.predict(X_test)

# Evaluate the model

print("\n========== NAIVE BAYES RESULTS ==========")

print(
    "Naive Bayes Accuracy:",
    accuracy_score(y_test, y_pred_nb)
)

print("\nClassification Report:")
print(classification_report(y_test, y_pred_nb))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_nb))

# ============================================================
# STEP 11: HANDLING IMBALANCED DATA USING SMOTE
# ============================================================

from imblearn.over_sampling import SMOTE

# Apply SMOTE only to training data

oversample = SMOTE(random_state=42)

X_sampled, y_sampled = oversample.fit_resample(
    X_train,
    y_train
)

print("\nClass distribution after SMOTE:")
print(y_sampled.value_counts())

# Visualize balanced dataset

sns.countplot(x=y_sampled)
plt.title("Class Distribution After SMOTE")
plt.show()

# ============================================================
# STEP 12: DECISION TREE USING SMOTE DATA
# ============================================================

# Create Decision Tree classifier

classifier = DecisionTreeClassifier(
    random_state=42
)

# Fit the model

classifier.fit(X_sampled, y_sampled)

# Predict the test set result

y_pred = classifier.predict(X_test)

# ============================================================
# STEP 13: EVALUATE THE MODEL
# ============================================================

print("\n========== SMOTE DECISION TREE RESULTS ==========")

print(
    "Confusion Matrix:",
    confusion_matrix(y_test, y_pred)
)

print(
    "Accuracy Score:",
    accuracy_score(y_test, y_pred)
)

print(
    "Precision Score:",
    precision_score(y_test, y_pred)
)

print(
    "Recall Score:",
    recall_score(y_test, y_pred)
)

print(
    "F-beta Score:",
    fbeta_score(y_test, y_pred, beta=1)
)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# ============================================================
# STEP 14: VISUALIZE DECISION TREE
# ============================================================

plt.figure(figsize=(20, 10))

tree = plot_tree(
    decision_tree=classifier,
    feature_names=X.columns,
```

```
    class_names=['0', '1'],
    filled=True
)

plt.title("Decision Tree After SMOTE")

plt.savefig("Decision Tree.png")

plt.show()

# ============================================================
# STEP 15: MODEL COMPARISON
# ============================================================

print("\n========== MODEL COMPARISON ==========")

print(
    "Decision Tree Accuracy:",
    accuracy_score(y_test, y_pred_dt)
)

print(
    "Naive Bayes Accuracy:",
    accuracy_score(y_test, y_pred_nb)
)

print(
    "SMOTE Decision Tree Accuracy:",
    accuracy_score(y_test, y_pred)
)

print("\n========== PROGRAM COMPLETED ==========")
```

Viva questions to practise

What is classification? How does a Decision Tree work? State Bayes' theorem. Why is Naïve Bayes called naïve? Define accuracy, precision, recall and F1-score.

Experiment 7: Clustering Techniques – K-Means and Hierarchical Clustering

Aim

To implement K-Means and Hierarchical Clustering, use Elbow Method and Dendrogram, and analyze the clusters.

Theory in simple words

Extra concepts / quick revision

Clustering is an unsupervised learning method. It groups similar data points when the correct group labels are not already provided. Customer segmentation is a common example. K-Means divides data into K clusters. First, it places K centroids. Each point is assigned to its nearest centroid. The centroids are recalculated, and assignment is repeated until the groups stop changing or the algorithm converges. The Elbow Method runs K-Means for different K values and plots WCSS (Within-Cluster Sum of Squares). WCSS measures how close points are to their cluster centroid. The point where the curve begins to bend (the elbow) is a possible K value. Hierarchical Agglomerative Clustering starts with every point as its own cluster and repeatedly merges the closest clusters. A dendrogram is a tree diagram showing these merges. A horizontal cut through the dendrogram suggests the number of clusters. Feature scaling is important because clustering uses distances. Silhouette score measures how well points fit their own cluster compared with other clusters; higher values (up to 1) generally indicate better-separated clusters. This practical uses Mall Customers data, applies K-Means and hierarchical clustering, and visualizes customer groups.

Algorithm / Procedure

1. Load Mall Customers CSV. 2. Inspect, clean and scale features. 3. Use Elbow Method to inspect K. 4. Fit K-Means and visualize clusters. 5. Draw dendrogram. 6. Fit hierarchical clustering and compare.

Complete Python Code

```
# ============================================================
# AIML GROUP C - ASSIGNMENT 1
# CLUSTERING TECHNIQUES
# K-MEANS AND HIERARCHICAL CLUSTERING
# ============================================================

# STEP 1: IMPORTING REQUIRED LIBRARIES

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

warnings.filterwarnings('ignore')

# ============================================================
# STEP 2: LOADING THE DATASET
# ============================================================

df = pd.read_csv("Mall_Customers.csv")

# Reading first rows
print("\nFirst 5 Rows:")
print(df.head())

# Checking dimensions
print("\nShape of Dataset:")
print(df.shape)

# Checking data types
print("\nData Types:")
print(df.dtypes)

# Renaming column
df.rename(columns={'Genre': 'Gender'}, inplace=True)

# Printing information
print("\nDataset Information:")
df.info()

# Descriptive statistics
print("\nDescriptive Statistics:")
print(df.describe())

# Checking missing values
print("\nMissing Values:")
print(df.isna().sum())

# Checking total missing values
print("\nTotal Missing Values:")
print(df.isna().sum().sum())

# Checking duplicates
print("\nDuplicate Values:")
print(df.duplicated().sum())

# Checking unique values
print("\nUnique Values:")
print(df.nunique())
```

```
# Dropping CustomerID column
df.drop(columns=['CustomerID'], inplace=True)

print("\nDataset After Dropping CustomerID:")
print(df.head())

# ========================================================
# STEP 3: EXPLORATORY DATA ANALYSIS (EDA)
# ========================================================

# Checking outliers
plt.figure(figsize=(10, 5))
sns.boxplot(data=df.select_dtypes(include=np.number))
plt.title("Checking Outliers")
plt.show()

# Distribution of Annual Income
plt.figure(figsize=(7, 5))
sns.histplot(df['Annual Income (k$)'], kde=True)
plt.title("Distribution of Annual Income")
plt.show()

# Distribution of Age
plt.figure(figsize=(7, 5))
sns.histplot(df['Age'], kde=True)
plt.title("Distribution of Age")
plt.show()

# Distribution of Spending Score
plt.figure(figsize=(7, 5))
sns.histplot(df['Spending Score (1-100)'], kde=True)
plt.title("Distribution of Spending Score")
plt.show()

# Distribution of Gender
plt.figure(figsize=(7, 5))
sns.countplot(x='Gender', data=df)
plt.title("Distribution of Gender")
plt.show()

# Relationship between Annual Income, Spending Score and Gender
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=df,
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='Gender'
)
plt.title("Annual Income vs Spending Score")
plt.show()

# ========================================================
# STEP 4: PREPROCESSING THE DATA BY SCALING FEATURES
# ========================================================

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

df.iloc[:, 1:] = scaler.fit_transform(df.iloc[:, 1:])

print("\nScaled Dataset:")
print(df.head(3))

# ========================================================
# STEP 5: CHECKING CORRELATION AMONG FEATURES
# ========================================================

print("\nCorrelation Matrix:")
print(df.corr(numeric_only=True))

plt.figure(figsize=(8, 5))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

# ========================================================
# STEP 6: APPLYING LABEL ENCODER
# ========================================================

from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df["Gender"] = le.fit_transform(df["Gender"])

print("\nDataset After Label Encoding:")
print(df.head(3))

# ========================================================
# STEP 7: FINDING OPTIMAL NUMBER OF CLUSTERS
# USING ELBOW METHOD
# ========================================================

from sklearn.cluster import KMeans

wcss_list = []

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(df)
    wcss_list.append(kmeans.inertia_)

print("\nWCSS Values:")
```

```
print(wcss_list)

# Plotting Elbow Method
plt.figure(figsize=(8, 5))
sns.lineplot(x=range(2, 11), y=wcss_list, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()

# ============================================================
# STEP 8: SILHOUETTE SCORE
# ============================================================

from sklearn.metrics import silhouette_score

silhouette_scores = []

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(df)

    labels = kmeans.predict(df)

    # Calculate Silhouette Score
    score = silhouette_score(df, labels)
    silhouette_scores.append(score)

print("\nSilhouette Scores:")
print(silhouette_scores)

# Plotting Silhouette Score
plt.figure(figsize=(8, 5))
sns.lineplot(
    x=range(2, 11),
    y=silhouette_scores,
    marker='o'
)
plt.title("Silhouette Score")
plt.xlabel("Number of Clusters")
plt.ylabel("Silhouette Score")
plt.show()

# ============================================================
# STEP 9: APPLYING K-MEANS CLUSTERING ALGORITHM
# ============================================================

num_clusters = 5

kmeans = KMeans(
    n_clusters=num_clusters,
    random_state=42,
    n_init=10
)

df['cluster'] = kmeans.fit_predict(df)

print("\nK-Means Clustering Results:")
print(df.head(3))

# ============================================================
# STEP 10: VISUALIZING RESULTING CLUSTERS
# ============================================================

plt.figure(figsize=(8, 5))

sns.scatterplot(
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='cluster',
    data=df,
    palette='viridis'
)

plt.title("K-Means Clustering")
plt.show()

# ============================================================
# STEP 11: APPLYING HIERARCHICAL CLUSTERING
# ============================================================

from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# Use original features without the K-Means cluster label
X = df.drop(columns=['cluster'])

# Compute Linkage Matrix
matrix = linkage(X, method='ward')

# Plot the Dendrogram
plt.figure(figsize=(10, 7))

dendrogram(
    matrix,
    orientation='top',
    distance_sort='descending',
    show_leaf_counts=True
)

plt.title('Dendrogram')
plt.xlabel('Sample Index')
plt.ylabel('Distance')
plt.show()
```

```
# ============================================================
# STEP 12: APPLY AGGLOMERATIVE CLUSTERING
# ============================================================

# Set the number of clusters
num_clusters = 4

agg_clustering = AgglomerativeClustering(
    n_clusters=num_clusters,
    linkage='ward'
)

clusters = agg_clustering.fit_predict(X)

# Add cluster labels to original data
df['Cluster1'] = clusters

print("\nHierarchical Clustering Results:")
print(df.head(3))

# ============================================================
# STEP 13: VISUALIZING HIERARCHICAL CLUSTERS
# ============================================================

plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='Cluster1',
    palette='viridis'
)

plt.title("Hierarchical Clustering")
plt.show()

# ============================================================
# CONCLUSION
# ============================================================

print("\nConclusion:")
print(
    "The clustering analysis successfully segmented customers "
    "into distinct groups, revealing valuable insights into "
    "their spending behavior."
)

print("\nClustering Assignment Completed Successfully!")
```

Viva questions to practise

What is clustering? Is it supervised? What is a centroid? Define WCSS. What does the elbow indicate? What is a dendrogram? Why scale data?

Experiment 8: Dimensionality Reduction using Principal Component Analysis (PCA)

Aim

To apply PCA on a high-dimensional dataset and compare feature count, visualization and classification performance.

Theory in simple words

Extra concepts / quick revision

A dataset may have many features (columns). Too many features can increase computation, make visualization difficult and sometimes include repeated information. Dimensionality reduction reduces the number of features while trying to keep useful information. Principal Component Analysis (PCA) is an unsupervised technique. It creates new features called principal components. The first principal component captures the largest possible amount of variation in the data. The next component captures the largest remaining variation and is at right angles (uncorrelated) to the first. PCA is affected by feature scale, so StandardScaler is normally applied first. Explained variance ratio tells how much of the dataset's variation each component keeps. For example, 0.95 means the selected components retain 95% of the variance. The practical uses the Breast Cancer dataset. It compares the number of original features with the reduced features, plots the first two components and compares Logistic Regression accuracy before and after PCA. Important: split the data first. Fit the scaler and PCA using training data only, then transform test data. This avoids data leakage.

Algorithm / Procedure

1. Load Breast Cancer data. 2. Split into train and test. 3. Standardize features. 4. Apply PCA and inspect explained variance. 5. Visualize two components. 6. Compare Logistic Regression before and after PCA.

Complete Python Code

```
# ============================================================
# AIML ASSIGNMENT 2
# DIMENSIONALITY REDUCTION USING PCA
# ============================================================

# 1. Import Necessary Libraries

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split


# 2. Load the Dataset

data = load_breast_cancer()

X = data.data
y = data.target

print("Original shape:", X.shape)

# Convert to DataFrame with feature names

df = pd.DataFrame(X, columns=data.feature_names)
df['target'] = y

print("\nDataset:")
print(df.head())


# 3. Split Dataset into Training and Testing Sets

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)


# 4. Standardize the Features

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nStandardized Features:")
print(X_train_scaled)


# 5. Calculate Covariance Structure
```

```
cov_matrix = np.cov(
    X_train_scaled,
    rowvar=False
)

print("\nCovariance Matrix:")
print(cov_matrix)


# 6. Apply PCA (2 Components)

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_train_scaled)

print("\nBefore PCA:", X_train_scaled.shape)
print("After PCA:", X_pca.shape)


# 7. Explained Variance

print("\nExplained Variance:")
print(pca.explained_variance_ratio_)

total_variance = sum(pca.explained_variance_ratio_)

print("Variance retained:", total_variance)


# 8. Retaining 95% Variance

pca_95 = PCA(n_components=0.95)

X_train_pca = pca_95.fit_transform(X_train_scaled)
X_test_pca = pca_95.transform(X_test_scaled)

print("\nOriginal features:", X_train_scaled.shape[1])
print("Reduced features:", X_train_pca.shape[1])


# 9. Visualization Using First Two PCs

plt.figure(figsize=(8, 6))

plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=y_train,
    cmap='viridis'
)

plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA Visualization")
plt.grid(True)
plt.show()


# 10. Classification Performance: Before PCA

start = time.time()

model = LogisticRegression(max_iter=5000)

model.fit(X_train_scaled, y_train)

pred = model.predict(X_test_scaled)

accuracy_before = accuracy_score(y_test, pred)

end = time.time()

time_before = end - start

print("\nClassification Before PCA:")
print("Accuracy without PCA:", accuracy_before)
print("Execution time without PCA:", time_before, "seconds")


# 11. Classification Performance: After PCA

start = time.time()

model_pca = LogisticRegression(max_iter=5000)

model_pca.fit(X_train_pca, y_train)

pred_pca = model_pca.predict(X_test_pca)

accuracy_after = accuracy_score(y_test, pred_pca)

end = time.time()

time_after = end - start

print("\nClassification After PCA:")
print("Accuracy with PCA:", accuracy_after)
print("Execution time with PCA:", time_after, "seconds")


# 12. Final Comparison

print("\n========== FINAL COMPARISON ==========")
```

```
print("Original Features:", X_train_scaled.shape[1])
print("Reduced Features:", X_train_pca.shape[1])

print("\nAccuracy Before PCA:", accuracy_before)
print("Accuracy After PCA:", accuracy_after)

print("\nTime Before PCA:", time_before, "seconds")
print("Time After PCA:", time_after, "seconds")

print("\n========== PROGRAM COMPLETED ==========")
```

Viva questions to practise

What is dimensionality reduction? Expand PCA. What is a principal component? Why standardize first? What does explained variance mean? What is data leakage?

Experiment 9: Advanced Machine Learning and Model Optimization

Aim

To compare SVM, Random Forest and AdaBoost using cross-validation and Grid Search optimization.

Theory in simple words

Extra concepts / quick revision

This practical compares Support Vector Machine (SVM), Random Forest and AdaBoost classifiers. SVM finds a decision boundary that separates classes with the largest possible margin. A kernel (such as linear or RBF) allows SVM to model different types of boundaries. SVM is sensitive to feature scale, so scaling is commonly used. Random Forest combines many decision trees. Each tree is trained using a sample of the data and a random subset of features. Combining trees usually improves stability and reduces overfitting compared with one tree. AdaBoost means Adaptive Boosting. It trains weak learners one after another. Later learners give more attention to records that earlier learners classified incorrectly. Their outputs are combined into a stronger model. Cross-validation evaluates a model several times using different training and validation folds. In 5-fold cross-validation, data is divided into five parts; each part is used once for validation while the others are used for training. Grid Search tests specified hyperparameter combinations and selects the combination with the best cross-validation score. Hyperparameters are settings chosen before training, such as SVM C/kernel or the number of trees. The practical compares accuracy, precision, recall and F1-score before and after optimization. The test set is reserved for final evaluation.

Algorithm / Procedure

1. Load Breast Cancer data. 2. Split into train and test. 3. Train SVM, Random Forest and AdaBoost. 4. Evaluate with 5-fold cross-validation. 5. Tune using GridSearchCV. 6. Compare before/after results.

Complete Python Code

```
# ============================================================
# GROUP C - ASSIGNMENT 3
# ADVANCED MACHINE LEARNING AND MODEL OPTIMIZATION
# SVM, RANDOM FOREST AND ADABOOST
# ============================================================

# 1. IMPORT NECESSARY LIBRARIES

import numpy as np
import pandas as pd

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

# ============================================================
# 2. LOAD THE DATASET
# ============================================================

# Load Breast Cancer Wisconsin Dataset

data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

print("Dataset Loaded Successfully!")
print("Dataset Shape:", X.shape)

# ============================================================
# 3. SPLIT THE DATASET
# ============================================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining Data:", X_train.shape)
print("Testing Data:", X_test.shape)

# ============================================================
# 4. INITIALIZE CLASSIFICATION MODELS
# ============================================================

models = {
```

```
    "SVM": Pipeline([
        ("scaler", StandardScaler()),
        ("svm", SVC())
    ]),

    "Random Forest": RandomForestClassifier(
        random_state=42
    ),

    "AdaBoost": AdaBoostClassifier(
        random_state=42
    )
}

# ============================================================
# 5. TRAIN MODELS AND PERFORM CROSS-VALIDATION
# ============================================================

baseline_results = []
cv_results = {}

print("\n========== BEFORE OPTIMIZATION ==========")

for name, model in models.items():

    # 5-Fold Cross Validation
    scores = cross_val_score(
        model,
        X_train,
        y_train,
        cv=5,
        scoring="accuracy"
    )

    cv_results[name] = scores.mean()

    # Train the model
    model.fit(X_train, y_train)

    # Predictions
    y_pred = model.predict(X_test)

    # Evaluation Metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)

    baseline_results.append({
        "Model": name,
        "CV Accuracy": scores.mean(),
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1
    })

    print("\nModel:", name)
    print("Cross Validation Accuracy:", round(scores.mean(), 4))
    print("Accuracy:", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall:", round(recall, 4))
    print("F1-Score:", round(f1, 4))


# ============================================================
# 6. HYPERPARAMETER TUNING USING GRID SEARCH
# ============================================================

param_grids = {

    "SVM": {
        "svm__C": [0.1, 1, 10],
        "svm__kernel": ["linear", "rbf"],
        "svm__gamma": ["scale", "auto"]
    },

    "Random Forest": {
        "n_estimators": [50, 100, 200],
        "max_depth": [None, 5, 10],
        "min_samples_split": [2, 5]
    },

    "AdaBoost": {
        "n_estimators": [50, 100, 150],
        "learning_rate": [0.01, 0.1, 1.0]
    }
}

optimized_models = {}
optimized_results = []

print("\n========== AFTER OPTIMIZATION ==========")

for name, model in models.items():

    print("\nOptimizing:", name)

    grid_search = GridSearchCV(
        estimator=model,
        param_grid=param_grids[name],
        cv=5,
        scoring="accuracy",
        n_jobs=-1
```

```
    )

    # Fit Grid Search
    grid_search.fit(X_train, y_train)

    # Best Model
    best_model = grid_search.best_estimator_

    optimized_models[name] = best_model

    # Predictions
    y_pred = best_model.predict(X_test)

    # Evaluation Metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)

    optimized_results.append({
        "Model": name,
        "CV Accuracy": grid_search.best_score_,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1
    })

    print("Best Parameters:", grid_search.best_params_)
    print("Best CV Accuracy:", round(grid_search.best_score_, 4))
    print("Accuracy:", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall:", round(recall, 4))
    print("F1-Score:", round(f1, 4))


# ============================================================
# 7. COMPARE MODEL PERFORMANCE
# ============================================================

before_df = pd.DataFrame(baseline_results)
after_df = pd.DataFrame(optimized_results)

print("\n========== PERFORMANCE BEFORE OPTIMIZATION ==========")
print(before_df.to_string(index=False))

print("\n========== PERFORMANCE AFTER OPTIMIZATION ==========")
print(after_df.to_string(index=False))


# ============================================================
# 8. COMBINED COMPARISON TABLE
# ============================================================

comparison = pd.concat(
    [
        before_df.set_index("Model").add_prefix("Before "),
        after_df.set_index("Model").add_prefix("After ")
    ],
    axis=1
)

print("\n========== FINAL COMPARISON TABLE ==========")
print(comparison.round(4).to_string())


# ============================================================
# 9. IDENTIFY THE BEST PERFORMING MODEL
# ============================================================

best_index = after_df["Accuracy"].idxmax()

best_model_name = after_df.loc[best_index, "Model"]
best_accuracy = after_df.loc[best_index, "Accuracy"]

print("\n========== BEST PERFORMING MODEL ==========")

print("Best Model:", best_model_name)
print("Best Test Accuracy:", round(best_accuracy, 4))


# ============================================================
# 10. CONCLUSION
# ============================================================

print("\n========== CONCLUSION ==========")

print(
    "SVM, Random Forest and AdaBoost algorithms were "
    "implemented successfully."
)

print(
    "Five-fold cross-validation and Grid Search were "
    "used for model optimization."
)

print(
    "The models were evaluated using Accuracy, Precision, "
    "Recall and F1-Score."
)

print(
    "The best performing optimized model is:",
```

```
    best_model_name
)

print("\nProgram Executed Successfully!")
```

Viva questions to practise

What is SVM margin? What is Random Forest? What is boosting? What is cross-validation? What is a hyperparameter? What does GridSearchCV do?

Experiment 10: AI/ML-Based Real-World Application – Customer Segmentation

Aim

To build an AI/ML-based customer segmentation application and document its data, model, evaluation and results.

Theory in simple words

Extra concepts / quick revision

A real-world AI/ML application follows a complete workflow: define the problem, collect a dataset, preprocess it, develop a model, evaluate it, optimize it and document the result. Problem definition: a shopping mall wants to group customers according to age, annual income and spending score. The groups can help the business plan suitable offers and marketing campaigns. Dataset: Mall_Customers.csv. Each row represents a customer. CustomerID is only an identifier and should not be used as a meaningful customer feature. Preprocessing: select useful columns, handle missing values and scale numeric features. Scaling is important because distance-based K-Means can be dominated by columns with larger numeric ranges. Model: K-Means clustering groups customers into K segments. The Elbow Method helps choose K by observing WCSS. The code uses K=5 as a practical segmentation choice and reports a silhouette score. Evaluation and analysis: inspect the number of customers in each cluster, compare average age/income/spending score and visualize annual income against spending score. The generated cluster labels are saved to a new CSV file. Optimization: try different K values, feature combinations and scaling choices, then compare WCSS and silhouette score. The clusters are exploratory groups, not guaranteed business categories.

Algorithm / Procedure

1. Define customer segmentation problem. 2. Load Mall Customers data. 3. Select and scale features. 4. Use Elbow Method. 5. Train K-Means. 6. Evaluate and interpret clusters. 7. Save results and document findings.

Complete Python Code

```
# ============================================================
# PRACTICAL 10 - AI/ML REAL-WORLD APPLICATION
# CUSTOMER SEGMENTATION USING K-MEANS CLUSTERING
# Dataset: Mall_Customers.csv
# ============================================================

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# 1. LOAD DATASET
df = pd.read_csv("Mall_Customers.csv")
print("First five rows:")
print(df.head())
print("\nDataset shape:", df.shape)
print("\nMissing values:\n", df.isnull().sum())

# 2. PREPROCESS DATA
# CustomerID is only an identifier, so it is not used for clustering.
# Select meaningful numeric customer attributes.
features = df[["Age", "Annual Income (k$)", "Spending Score (1-100)"]].copy()
features = features.fillna(features.median())

# 3. SCALE FEATURES
scaler = StandardScaler()
X_scaled = scaler.fit_transform(features)

# 4. FIND NUMBER OF CLUSTERS USING ELBOW METHOD
wcss = []
for k in range(1, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    wcss.append(model.inertia_)

plt.plot(range(1, 11), wcss, marker="o")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()

# 5. TRAIN K-MEANS MODEL
# K=5 is a commonly used segmentation choice for this dataset.
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(X_scaled)

# 6. EVALUATE CLUSTERS
score = silhouette_score(X_scaled, df["Cluster"])
print("\nSilhouette Score:", round(score, 3))
print("\nCustomer count in each segment:\n", df["Cluster"].value_counts().sort_index())

# 7. ANALYZE SEGMENTS
segment_summary = df.groupby("Cluster")[
    ["Age", "Annual Income (k$)", "Spending Score (1-100)"]
].mean().round(2)
print("\nAverage profile of each customer segment:\n", segment_summary)
```

```
# 8. VISUALIZE CUSTOMER SEGMENTS
plt.figure(figsize=(8, 5))
plt.scatter(
    df["Annual Income (k$)"],
    df["Spending Score (1-100)"],
    c=df["Cluster"],
    cmap="viridis"
)
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Customer Segmentation using K-Means")
plt.colorbar(label="Cluster")
plt.show()

# 9. SAVE RESULT
df.to_csv("Customer_Segmentation_Results.csv", index=False)
print("\nResults saved to Customer_Segmentation_Results.csv")
print("Customer segmentation completed successfully.")
```

Viva questions to practise

What problem does the project solve? Which dataset and features are used? Why remove CustomerID? Why scale? What is K-Means? How are clusters evaluated and interpreted?

Final Exam Revision – Important Differences

Topic	Remember this
BFS vs DFS	BFS: queue, level-wise, shortest unweighted path. DFS: stack, depth-wise, not guaranteed shortest.
Forward vs Backward Chaining	Forward: known facts → conclusions. Backward: goal → required facts.
Scaling vs Encoding	Scaling changes numeric range. Encoding converts categories/text into numbers.
Regression vs Classification	Regression predicts a continuous value. Classification predicts a class label.
Supervised vs Unsupervised	Supervised learns from labelled examples. Unsupervised finds patterns without target labels.
K-Means vs Hierarchical	K-Means needs K. Hierarchical clustering can be explored with a dendrogram.
PCA vs Feature Selection	PCA creates new combined components. Feature selection keeps selected original features.
Cross-validation vs Grid Search	Cross-validation evaluates stability. Grid Search tests hyperparameter combinations.

Dataset and code notes

The code is based on the scripts and datasets supplied in ALML Lab.zip, with corrections or additions where needed to match the official experiment list. In Experiment 5, the script reads california_housing.csv (the actual filename in the archive). Experiment 3 now demonstrates both chaining methods. Experiment 6 focuses on the two classifiers required by the university list and avoids an optional SMOTE dependency. Experiment 10 is a complete Customer Segmentation example, one of the application choices explicitly listed in the university document.

For practical execution, run each experiment separately. If a library or CSV file is missing, install the library or upload the matching dataset. Numerical results may vary slightly with software versions and train-test splits.